Universidad Nacional
de La Matanza

Algoritmos y Estructuras de Datos

Trabajo Práctico Caravana del Desierto

Comisiones 02-3300 y 03-3600

Versión 1.0

Requerimiento

Se debe diseñar e implementar un juego denominado Caravana del Desierto. La historia del juego es la siguiente:

Una caravana intenta atravesar una antigua ruta comercial para llegar a una ciudad refugio antes de que se agoten sus recursos.

A lo largo del trayecto existen tesoros, provisiones, oasis, tormentas de arena y bandidos.

El desierto es hostil e impredecible. En cada turno el jugador arrojará un dado virtual que generará un valor aleatorio entre 1 y 6. El valor obtenido determinará exactamente la cantidad de posiciones que deberá desplazarse en ese turno.

Los bandidos se desplazan automáticamente tratando de interceptar al jugador.

Cada decisión cuenta. La caravana debe avanzar. El desierto no perdona errores.

Reglas del juego

El jugador (**J**) debe desplazarse sobre una ruta desértica persiguiendo el objetivo de llegar desde el Campamento Inicial (**I**) hasta la Ciudad Refugio (**S**), evitando ser interceptado por los Bandidos (**B**) y aprovechando los recursos distribuidos a lo largo del recorrido. Para los bandidos es posible desplazarse hacia atrás y hacia adelante inclusive a través de la entrada y la salida, como si la ruta fuera un círculo. La partida finalizará cuando el jugador alcance la Ciudad Refugio o cuando pierda todas sus vidas antes de alcanzarla.

Si, al efectuar un lanzamiento del dado, el jugador sobrepasara la Ciudad Refugio, deberá continuar su desplazamiento retrocediendo la cantidad de posiciones sobrantes, de modo tal que el movimiento total realizado sea exactamente igual al valor obtenido en el dado.

Criterios generales:

- Ruta del desierto: la ruta se encontrará compuesta por una secuencia de posiciones enlazadas entre sí. Desde el punto de vista de su representación en la computadora, deberá modelarse mediante una Lista Circular Doblemente Enlazada. Cada nodo de la lista representará una posición del recorrido.
- Campamento Inicial (**I**): es el punto desde el cual comienza la partida.
- Ciudad Refugio (**S**): es la meta del recorrido y representa la salida del juego.

- Jugador (**J**): es el componente animado a través de los eventos producidos por el usuario. En cada turno arrojará un dado virtual y deberá desplazarse exactamente la cantidad de posiciones indicada por el valor obtenido. El jugador podrá decidir si realiza dicho desplazamiento hacia adelante o hacia atrás sobre la ruta. Cada acción consume un turno. El movimiento se considerará resuelto únicamente en la posición de destino; por lo tanto, solo se aplicarán sobre el jugador los efectos correspondientes al punto en el que finalmente cae, sin que las posiciones intermedias produzcan consecuencias.
- Dado virtual: debe generar un valor aleatorio entero entre 1 y 6 inclusive. Ese valor determinará exactamente la cantidad de posiciones que deberá desplazarse el jugador en el turno correspondiente.
- Premios (**P**): se encuentran distribuidos a lo largo de la ruta. Cuando el jugador capture un premio obtendrá 1 punto.
- Vidas extra (**V**): cuando el jugador capture una Vida Extra se incrementará en uno el número de vidas disponibles para continuar jugando.
- Oasis (**O**): cuando el jugador llegue a un oasis obtendrá protección para el turno siguiente. Durante ese turno no podrá perder vidas a causa de bandidos ni tormentas.
- Tormentas de arena (**T**): cuando el jugador caiga en una tormenta perderá el siguiente turno. Si el jugador se encontrara protegido por un oasis, la tormenta no producirá efecto.
- Bandidos (**B**): se desplazan automáticamente sobre la ruta. En cada turno de la computadora, cada bandido podrá avanzar o retroceder sobre la lista según el criterio definido por el equipo de diseño. Un bandido afectará al jugador únicamente si, al finalizar el movimiento del jugador o del propio bandido, ambos quedan ubicados en la misma posición. En ese caso, el jugador perderá una vida, volverá al Campamento Inicial y el bandido que lo haya interceptado será eliminado de la ruta. Si el jugador ya no posee vidas, la partida finalizará.

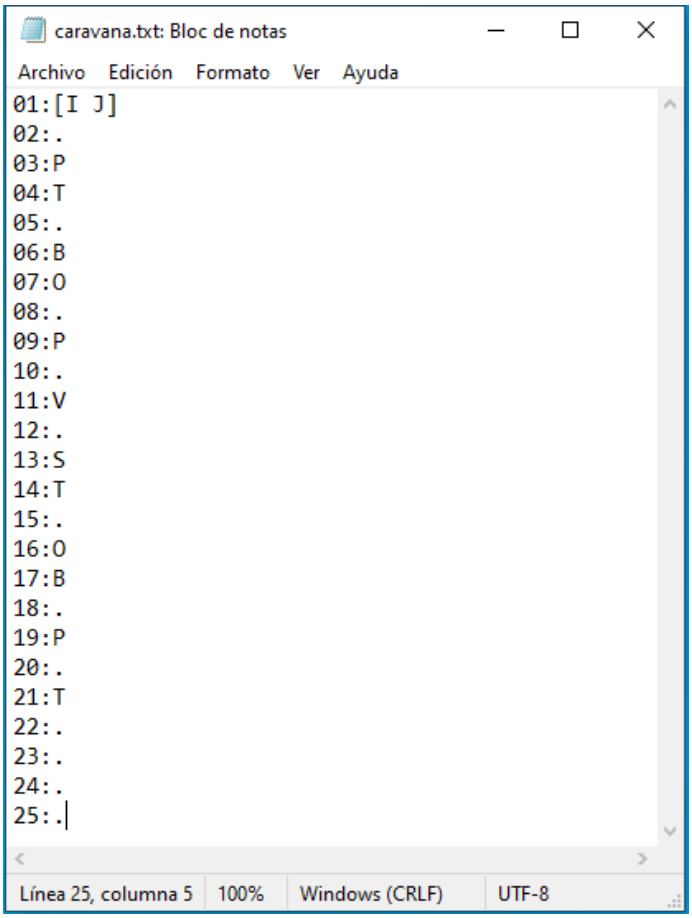
Consigna

a) Se debe implementar el juego Caravana del Desierto en lenguaje C.
b) Configuraciones iniciales: mediante un archivo de texto denominado **config.txt** será posible establecer los siguientes parámetros:
    - cantidad_posiciones: número máximo de posiciones en el tablero.
    - vidas_inicio: cantidad de vidas iniciales.
    - maximo_bandidos: número de bandidos a distribuir en el tablero.
    - maximo_premios: número de premios a distribuir en el tablero.
    - maximo_vidas_extra: cantidad de vidas extra a distribuir en el tablero.
    - maximo_oasis: número de oasis a distribuir en el tablero.
    - maximo_tormentas: número de tormentas a distribuir en el tablero.
c) Al iniciar el juego, se debe generar el tablero en un archivo de texto denominado **caravana.txt** a partir de los parámetros de configuración provenientes del archivo **config.txt** descripto en el ítem anterior. La representación de los componentes del juego debe realizarse con los siguientes caracteres ASCII:
    - Jugador: **J**
    - Inicio: **I**
    - Salida: **S**
    - Premio: **P**
    - Vida extra: **V**
    - Oasis: **O**
    - Tormenta: **T**
    - Bandido: **B**
    - Posición vacía o ruta despejada: **.**
    - Casillero con jugador: **[J]**
    - Casillero con jugador y oasis: **[O J]**
    - Casillero con jugador y Tormenta: **[T J]**
    - etc.

La representación indicada precedentemente establece los caracteres básicos que deberán utilizarse para describir los componentes del juego. Los casilleros compuestos mostrados, tales como [J], [O J] o [T J], son únicamente ejemplos ilustrativos. El equipo de diseño podrá definir una notación consistente para representar otras combinaciones de elementos dentro de un mismo casillero, siempre que dicha convención sea clara, uniforme y se encuentre debidamente documentada.

Un ejemplo del archivo de texto que describe la caravana o con los siguientes valores en los parámetros:

- cantidad_posiciones:25
- vidas_inicio: 3
- maximo_bandidos: 2
- maximo_premios: 3
- maximo_vidas_extra: 1
- maximo_oasis: 2
- maximo_tormentas: 3



NOTA: Tener en cuenta que al iniciar una nueva partida debe crearse un **nuevo** escenario respetando los parámetros iniciales. Cada escenario que se genere debe tener una solución posible.

d) Los movimientos del jugador y de la computadora (bandidos) deben ser encolados. Luego se deben desencolar, y determinar el nuevo estado y actualizar la pantalla.

e) Se debe llevar un registro de los movimientos realizados por el jugador. Al finalizar la partida se debe mostrar el registro de movimientos. El formato será FX o BX, en donde FX indica adelante x casillas (**F**orward **X** casillas) y BX indica atrás x casillas (**B**ackward **X** casillas)

f) Se debe implementar una lógica para que la computadora haga sus movimientos. Los movimientos se verán reflejados sobre los bandidos, los cuales irán tras el jugador tratando de capturarlo.

g) El juego debe contar con una capa de gestión de datos en la que se hará el registro de usuarios, partidas, puntuaciones obtenidas y cantidad de movimientos realizados en cada partida. Al menos deberá contarse con dos archivos, uno de jugadores y otro de registro de partidas jugadas por los jugadores (un jugador puede tener ninguna, una o varias partidas jugadas). El archivo de jugadores deberá accederse a través de un índice implementado sobre un árbol de búsqueda binaria. El índice debe mantenerse actualizado y además se debe persistir sobre un archivo binario. Al iniciarse el juego, de existir registro de jugadores, ese registro debe accederse a través del índice persistido anteriormente.

h) El juego deberá contener un menú que permita: ver el ranking de jugadores, jugar una nueva partida, salir del juego. Si el equipo de diseño considera alguna otra opción es libertad de este implementarla. El ranking estará ordenado a partir de la cantidad de puntos obtenidos por cada jugador a lo largo de todas sus partidas registradas.

i) Al iniciar el juego debe indicarse el nombre del jugador. En caso de no existir en los registros se procederá a su alta.

j) El trabajo se encuentra abierto al agregado de características y/o funcionalidades que el equipo de diseño considere relevante. Estas modificaciones deben ser declaradas y documentadas. Al momento de la entrega del trabajo deben estar implementadas y en correcto funcionamiento.

Conformación del grupo de trabajo

Este trabajo práctico se realiza en equipos de hasta 4 personas, sin excepción. Los equipos deben ser informados a través de la plataforma MIeL con un mensaje en el foro “Trabajo Práctico” con plazo máximo hasta el día 20/04/2026 a las 23:59. Debe indicarse el nombre del equipo y sus integrantes (apellido, nombres y DNI).

Entrega

La entrega del trabajo práctico se realizará de manera individual. Cada integrante del grupo deberá enviar desde la sección prácticas de MIeL en la práctica “Trabajo Práctico” a todos los docentes, una copia del programa informando nuevamente en el mensaje los integrantes de su grupo.

El plazo máximo de entrega del trabajo práctico será el día 17/06/2026 a las 23:59 a través de la plataforma MIeL.

Los trabajos entregados después de la fecha indicada no serán tenidos en cuenta, por lo que el equipo obtendrá como nota un ausente en la entrega y en la defensa, teniendo que utilizar la fecha de recuperatorio para regularizar su situación.

La entrega constará de un archivo con el siguiente formato:

TP_ALGORIMOS_2026_C1_0X-3Y00_{NOMBRE_DEL_GRUPO}.zip

En donde X e Y dependerán de la comisión a la que pertenezca el grupo.

No deberá incluir las carpetas ***bin*** **y** ***obj***.

En la entrega sobre la plataforma MIeL, se deberá adjuntar el archivo .zip y la URL del repositorio en GitHub en donde se encuentra versionado el código. El repositorio debe ser **público**.

Defensa

Habrá una defensa del trabajo practico durante las clases de los días 24/06/2026 y 08/07/2026. La defensa constará de una evaluación grupal e individual en la que los integrantes demostrarán autoría y conocimiento del producto realizado y entregado.

Condiciones mínimas para lograr la aprobación

- El producto entregado deberá cumplir con los requerimientos y las consignas indicadas en el trabajo.
- Debe compilar con 0 errores 0 warnings
- Código prolijo, dividido en funciones
- Proyecto dividido en archivos .h y .c.
- Funciones lo más genéricas posibles
- Nombres significativos de variables
- El desarrollo debe ser colaborativo y se pondrá de manifiesto a través de los commits propuestos por los integrantes en GitHub.